

Docker and PostgreSQL with Docker Compose

From zero to a running local database for Node.js apps

Learning Goals

- Understand Docker images, containers, and volumes
- Start PostgreSQL with Docker Compose
- Persist data between restarts
- Connect from your app using `DATABASE_URL`
- Debug common startup and auth issues

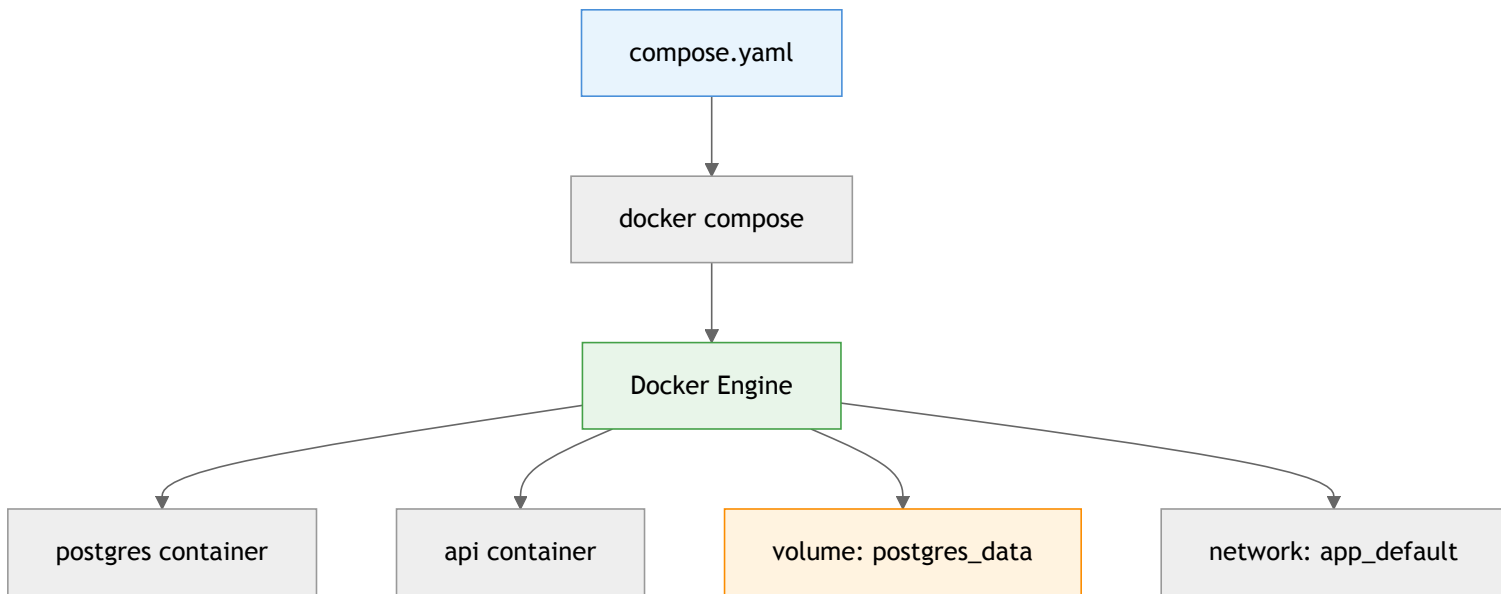
What Docker Is

- Docker is a platform for running applications in isolated containers
- Containers package app code plus runtime dependencies for consistent behavior
- Docker Engine builds, runs, and manages containers on your machine
- **Image**: blueprint for a container
- **Container**: running instance of an image
- **Volume**: persistent data outside container lifecycle
- **Network**: communication layer between containers

Compose orchestrates these pieces using one file.

What Docker Compose Is

- Docker Compose is a tool for defining multi-container applications
- You describe services, volumes, and networks in `compose.yaml`
- One command can start, stop, and rebuild the whole stack
- It standardizes local environments across a team



Why Use Compose for Postgres?

- Compose is infrastructure-as-code for local environments
- One command starts your database stack
- Consistent setup across your team
- Easy cleanup and rebuild
- Keep database config in version control

Ideal for local development and onboarding.

Prerequisites

1. Install Docker Desktop
2. Verify Docker is running
3. Create a project folder

```
# Check versions
docker --version
docker compose version
```

Project Files

```
my-api/  
  compose.yaml  
  .env
```

`compose.yaml` defines services.

`.env` stores local credentials (never commit real secrets).

compose.yaml (Postgres)

```
services:
  postgres:
    image: postgres:16
    container_name: academy-postgres
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: academy_db
      POSTGRES_USER: academy_user
      POSTGRES_PASSWORD: academy_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U academy_user -d academy_db"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  postgres_data:
```

Start and Verify

```
# Start in background
docker compose up -d

# Follow logs
docker compose logs -f postgres

# List running containers
docker compose ps
```

Container is ready when logs show `database system is ready to accept connections`.

Connect from Your App

```
DATABASE_URL="postgresql://academy_user:academy_password@localhost:5432/academy_db"
```

Use this in Prisma, Knex, or pg.

Test connection inside container

```
docker exec -it academy-postgres psql \  
-U academy_user -d academy_db
```

Quick SQL check:

```
SELECT current_database(), current_user;
```

Lifecycle Commands

```
# Start existing stack
docker compose up -d

# Stop containers
docker compose stop

# Stop + remove containers/network
docker compose down

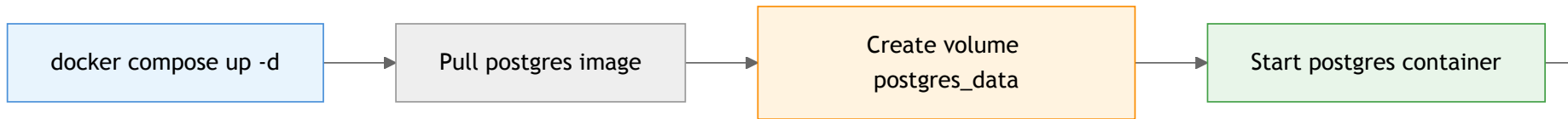
# Remove everything, including DB data
docker compose down -v
```

Use `down -v` only when you intentionally want a clean database.

Common Issues and Fixes

Problem	Fix
Port <code>5432</code> in use	Change host port: <code>"5433:5432"</code>
Auth failed	Verify user/password in <code>.env</code> and compose file
Data not reset	Run <code>docker compose down -v</code>
Service not ready	Check health and logs before app starts

Compose Startup Flow



Key Takeaways

- Compose gives repeatable local infrastructure
- Use volumes to keep Postgres data safe across restarts
- Use logs and healthchecks to debug startup problems
- Your app connects through a standard `DATABASE_URL`

Next: wire this database into Prisma migrations.